COMPUTER SCIENCE + COMPUTER ENGINEERING

A guided undergraduate-to-graduate route for games

04

Native-engineering reference

C++20 / C17 conventions and contracts

Types, ownership, invalidation, complexity, language semantics, resource budgets and troubleshooting.

PACKAGE 04 / EXPANDED VOLUME 03 / SEPTEMBER 28, 2026
Independent study, implementation, evidence and review.

Contents

Read contracts before syntax 3
A small syntax map 3
Numbers and representation 4
Ownership and exception safety 4
Common invalidation rules 4
Complexity with assumptions 5
The small language at a glance 6
Code map and validation scope 6
Troubleshooting without hiding the defect 7
Continuity with the original program 7
Primary reference directory 8
Reading by topic 10

C++ and native-engineering reference

Read contracts before syntax

This companion summarizes the selected C++20/C17 teaching environment and the bundle's own data contracts. It is not a substitute for the lessons or the language standard. C++ Core Guidelines are design guidance; CMake documentation describes build behavior; the algorithm and LLVM sources provide deeper primary study. Record documentation versions when continuing the course. [R11–R14]

A small syntax map

Form	Meaning in these examples	Important boundary
`T value`	Own a value/object	lifetime ends according to storage and scope
`const T& value`	Borrow an existing object for reading	does not generally extend its owner's life
`T* value`	Pointer, often a nullable borrow here	ownership is a contract, not determined by spelling
`std::unique_ptr<T>`	One owning handle	moves ownership; does not copy it
`std::optional<T>`	A value or explicit absence	absence is not automatically a parse error
`std::span<T>`	Contiguous borrowed range	length-aware, non-owning; indexing still needs a valid bound
`std::string_view`	Borrowed character range	characters may die or change independently
`std::move(x)`	Express an expiring-value category	selected move/copy operation does the work
`const`	Restricts mutation through that access path	not a general ownership or thread-safety guarantee
`noexcept`	Promise not to propagate exceptions	violation terminates; do not add it merely for speed
`requires` / concept	Constrain expressions/types	does not prove semantic laws such as transitivity

In declarations, `&` denotes a reference; in an expression such as `&object`, it obtains an address. `*pointer` accesses the referred object only when the pointer is valid for that access. `auto` deduces a type; it does not remove lifetime rules. `auto value=object` typically copies, while `auto& value=object` borrows. Be explicit when that difference is the point of the interface. [R11]

Numbers and representation

Inspect `sizeof`, `std::numeric_limits<T>::min()/max()`, `digits`, `max_digits10` and `is_iec559` rather than assuming native type widths. Exact-width integer types are used where available in this teaching target. Signed arithmetic overflow is undefined behavior; unsigned arithmetic wraps modulo its width, which can still violate your size or time contract. Check before adding/multiplying/converting. [P4-07]

For signed checked addition `a+b`, reject `b>0 && a>max-b`, or `b<0 && a<min-b`, before the sum. For nonnegative multiplication used as an allocation size, with nonzero `b`, compare `a > limit/b` before multiplying. Handle zero separately. The result of a cast cannot recover bits already discarded or repair arithmetic that already overflowed.

Floating-point operations round, and algebraically equivalent real expressions can have different intermediate rounding. A single absolute epsilon is not a universal equality policy. Use problem-scale-aware absolute/relative tolerances, finite-value checks and explicit units. Deterministic integer examples do not establish cross-machine floating-point replay. Do not use fast-math for the correctness baseline. [P4-02]

Little-endian serialization writes the least significant byte first. Native object layout includes possible padding and is not automatically a file/network format. The bridge defines its own eight-byte layout independently of native `sizeof`.

Ownership and exception safety

The owner releases a resource; a borrower does not. Prefer an immediately established owner after acquisition. A destructor performs cleanup on ordinary scope exit and relevant exception unwinding. Fully constructed members are cleaned up if later construction fails; the destructor of the incomplete enclosing object itself does not run. [R11: R, C, E]

The **no-throw guarantee** means the operation does not fail by propagating an exception. The **strong guarantee** means failure leaves observable state unchanged under the contract. The **basic guarantee** preserves valid invariants and prevents leaks but may change state. These guarantees attach to operations, not to a class name in the abstract.

Prepare a replacement in temporary owned state, validate it, then commit through a suitable nonthrowing swap/move. A wrapper with a raw unique handle must define or delete copy/move behavior deliberately. Rule-of-zero designs delegate ownership to standard owners; a low-level wrapper may need explicit special members. `shared_ptr` shares a lifetime but does not solve ownership cycles or provide automatic safe concurrent mutation of the object.

Common invalidation rules

Container/action	What to assume for this course
vector reallocation	all element pointers/references/iterators become invalid
vector erase	erased elements and positions at/after the erase are invalidated

Container/action	What to assume for this course
vector append without reallocation	existing element references remain, but the old end iterator changes
vector reserve	avoids reallocations only until capacity is exceeded; not a general lifetime fix
unordered_map rehash	iterators are invalidated; references/pointers to non-erased elements remain valid
destruction of owning container	its elements' lifetimes end regardless of spare capacity
logical ID lookup	identity can survive relocation if the ID/reuse policy is correct

Different containers and operations have different contracts. The table is not permission to hold a borrow forever. Recheck the exact operation and implementation-independent library contract when changing designs. The course's own APIs usually return copied snapshots so that callers do not retain element borrows across mutations. [R11; P4-08]

Complexity with assumptions

Operation/design	Cost statement	Qualification
Linear scan	O(n)	contiguous layout can have favorable constants
Sorted-array lower_bound	O(log n) comparisons	random access; ascending-order precondition
Doubling dynamic-array push	amortized O(1)	one growth push can move O(n) elements
Linked insertion after known node	O(1) linking	excludes locating/allocating the node
Fixed-capacity ring push/pop	O(1)	single-threaded, reject-on-full semantics
Hash table lookup	expected O(1), worst O(n)	depends on hash distribution and load policy
Binary heap push/pop	O(log n)	top is O(1); arbitrary membership is not binary search
BFS / FIFO Kahn topological sort	O(V+E)	adjacency lists; validation and output are counted separately as appropriate
Lazy-entry binary-heap Dijkstra	O((V+E) log(V+E)) conservative	nonnegative costs; stale records can accumulate up to edge-related work
0/1 knapsack DP	O(nB) time, O(B) space	pseudo-polynomial in numeric budget B

The lazy Dijkstra bound describes the supplied algorithm rather than promising an indexed decrease-key implementation. A* performance depends on heuristic quality and reopening behavior; never infer optimality solely from speed. Benchmarks compare complete equivalent work and record repeated samples. [R13]

The small language at a glance

Pipeline: **source -> located tokens -> owned AST -> tree evaluator**, or **AST -> postorder bytecode -> checked stack VM**. An optional folder transforms only successful constant subtrees. The base grammar supports nonnegative decimal integer literals, unary minus, parentheses and four binary arithmetic operators. Multiplication/division bind more tightly than addition/subtraction; each binary level is left-associative.

The value interval is [-10^9,+10^9]; division truncates toward zero. Unknown names, unary plus, decimal fractions, implicit multiplication and trailing tokens are not supported. This deliberate language differs from C++, Python and LLVM's tutorial language. All source positions are zero-based byte offsets.

Instruction	Input stack requirement	Stack effect
PUSH value	0	+1
NEG	1	0
ADD / SUB / MUL / DIV	2	-1

For binary operations, pop right first and left second; compute left op right. A complete expression must leave exactly one value. Default limits: source16,384 bytes; tokens4,096 including End; nodes4,096; depth64; instructions4,096; stack256; fuel4,096. Limits and semantics are specified in CAPSTONE_SPEC.pdf and implemented in `code/include/language.hpp`.

Constant folding may replace `2+3*4` with14, but not `(1/0)*0` with0. Preserve required errors and source context. A straight-line finite instruction list terminates under the bounded primitive-operation model; adding loops/callbacks changes that reasoning. Fuel is not a complete sandbox. [R14; P4-06]

Code map and validation scope

File	Main content
include/model.hpp	value snapshots, stable-ID store, bounded grid transition
include/resources.hpp	mock RAII owner, move and transactional replacement
include/structures.hpp	int-only buffer, ring, chained map, min-heap
include/algorithms.hpp	half-open search, stable sorts, bounded knapsack
include/graphs.hpp	BFS, topological order, Dijkstra/A* and reconstruction
include/language.hpp	bounded frontend, AST, bytecode VM, folding and finite-state model
include/check.hpp	always-active checks and controlled exception expectations
c_bridge/ge_c_api.h/.c	actual-C opaque ownership, status and field serialization
reference/layout_benchmark.cpp	narrow AoS/SoA update with checksum and raw samples

The code is educational and uses bounded, explicit domains rather than production-general containers or compilers. An int-only dynamic buffer is not a correct generic replacement for vector. The educational buffer and map deliberately disable copy/move rather than expose an incorrect moved-from invariant. Mock ownership counters must outlive their owners. The ring and C failure hooks are not concurrent algorithms. AST-consuming operations trust parser-produced structures. Read those preconditions before reusing a function in a wider context.

Troubleshooting without hiding the defect

Configure cannot find a compiler: establish a functioning C/C++ toolchain and use its configured terminal; do not lower the requested language level to conceal missing support. **A concept/span feature is unknown:** verify the actual C++20 compile command and standard library. **Undefined symbol:** check the matching declaration, definition and target source/library list; do not include a `.cpp` file as a repair.

A stale program runs after a failed build: stop and check exit status and executable path. **A starter reports TODO:** implement that contract, then add the remaining workbook cases; the initial failure is expected. **Release silently skips a test:** do not rely solely on assert when NDEBUG may be set; the included CHECK always executes.

Sanitizer configuration fails: use the compiler's official supported runtime/platform setup, record the limitation and run the other checks; do not state that sanitizers passed when they were unavailable. **Benchmark disagrees with your prediction:** verify equivalent work/checksum, build configuration, timing boundary and variance before changing the conclusion. Correctness is not optional for a fast result. [R12; P4-03; P4-04]

Sources and provenance

Continuity with the original program

This expansion was prepared for the September28,2026 curriculum edition. The original program manifest identifies Volume03 as C++, Algorithms and Language Implementation, Modules11–15, GateC and400–650 planning hours. Its lesson IDs, titles, outcomes, prerequisites, practical assignments and assigned readings are preserved in `source/ORIGINAL_VOLUME_03_MANIFEST.json`. The 60 original quiz questions and answers were recovered from the original `CURRICULUM.html`; the expansion adds40 application questions without replacing the originals.

Teaching explanations, worked examples, code, additional exercises, reviews, bridge material and project elaborations are original instructional synthesis for this bundle. Primary reading references support the underlying concepts. No external textbook, paper, compiler distribution, proprietary SDK or font file is included. Linked resources retain their own terms. Current documentation was consulted on September28,2026; historical course archives are intentionally used for durable foundations. Not every section of every historical source was re-reviewed in full.

The original assignments are preserved even where the starter supplies only a first contract. The full lab remains a learner requirement. Complete reference examples provide comparison evidence, not pre-completed module projects. The source JSON distinguishes original questions from additions.

Primary reference directory

R06 — 6.042J Mathematics for Computer Science

MIT OpenCourseWare. Proofs, graphs, counting and discrete probability.

https://ocw.mit.edu/courses/6-042j-mathematics-for-computer-science-spring-2015/

R11 — C++ Core Guidelines

C++ Core Guidelines editors. Resource safety, interfaces, classes and concurrency; guidance rather than the normative language standard.

https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines

R12 — CMake tutorial

Kitware. Build systems; use documentation matching the installed release.

https://cmake.org/cmake/help/latest/guide/tutorial/index.html

R13 — 6.006 Introduction to Algorithms

MIT OpenCourseWare. Data structures, graph algorithms and dynamic programming.

https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-fall-2011/

R14 — Kaleidoscope and LLVM tutorials

LLVM Project. Compiler frontends, intermediate representation and JIT compilation.

https://llvm.org/docs/tutorial/

R15 — 6.004 Computation Structures

MIT OpenCourseWare. Logic, processor design, memory and systems.

https://ocw.mit.edu/courses/6-004-computation-structures-spring-2017/

R16 — Operating Systems: Three Easy Pieces

Remzi and Andrea Arpaci-Dusseau. Virtualization, concurrency and persistence; book by its authors.

https://pages.cs.wisc.edu/~remzi/OSTEP/

R20 — 15-466/666 Computer Game Programming

Carnegie Mellon University. Runtime systems and game-production programming.

https://graphics.cs.cmu.edu/courses/15-466/

P4-01 — CMake compile features and language standards

Kitware. Target requirements, language standards and extensions. Use the documentation matching your CMake release.

https://cmake.org/cmake/help/latest/manual/cmake-compile-features.7.html

P4-02 — What Every Computer Scientist Should Know About Floating-Point Arithmetic

David Goldberg / Oracle documentation. Author paper reproduced in official numerical-computation documentation; rounding and non-associativity background.

https://docs.oracle.com/cd/E19957-01/806-3568/ncg_goldberg.html

P4-03 — AddressSanitizer

LLVM / Clang. Compiler/runtime setup and kinds of executed memory errors detected. Detection is not an all-input proof.

https://clang.llvm.org/docs/AddressSanitizer.html

P4-04 — UndefinedBehaviorSanitizer

LLVM / Clang. Runtime instrumentation and supported checks. Platform/toolchain support must be verified.

https://clang.llvm.org/docs/UndefinedBehaviorSanitizer.html

P4-05 — Language standards supported by GCC

GNU Compiler Collection. C/C++ standards and language modes. This course selects C17/C++20, not a newest-standard claim.

https://gcc.gnu.org/onlinedocs/gcc/Standards.html

P4-06 — Kaleidoscope: implementing a parser and AST

LLVM Project. Frontend architecture reading; the course language and integer semantics are independently specified and do not require LLVM libraries.

https://llvm.org/docs/tutorial/MyFirstLanguageFrontend/LangImpl02.html

P4-07 — Fundamental types [basic.fundamental]

C++ working-draft editors / eel.is rendering. Current working-draft view, not the fixed published C++20 edition. Consult only the stable integer/representation concepts used here; later additions are outside this course baseline.

https://eel.is/c++draft/basic.fundamental

P4-08 — Vector modifiers, capacity and unordered-container requirements

C++ working-draft editors / eel.is rendering. Companion sections: https://eel.is/c++draft/vector.capacity and https://eel.is/c++draft/unord.req. Used for operation-specific invalidation rules; current draft contains later features not used by the course.

https://eel.is/c++draft/vector.modifiers

Reading by topic

Use R11 for ownership, interfaces, special members, exception safety and source-file organization; R12/P4-01 for builds; R13 for arrays, hashing, heaps, sorting, graphs and dynamic programming; R14/P4-06 for language frontends and compiler structure. R06 supports discrete proof and computability reasoning, and R15 connects representations to later hardware study. R16/R20 are cross-volume reading pointers, not dependencies of the supplied build.

Read the exact assignment printed under each lesson. Record the section actually studied and at least one exercise or derivation in the journal. The narrow course implementation deliberately differs from some reading examples: bounded integers instead of general floating-point language values, a small straight-line stack machine instead of an LLVM JIT, and mock resources instead of a graphics API. Do not merge incompatible assumptions silently.